API Reference

This document describes every external API the Shrutlekh app calls: endpoint, auth, request/response shape, and the call site(s) in the codebase. Credential values themselves are never reproduced here — only which variable holds them and where that variable is defined.

---

1. BHASHINI / Dhruva — Streaming ASR (WebSocket)

Real-time speech-to-text over the mic or tab audio, used while a session is live.

- **Base URL:** `wss://dhruva-api.bhashini.gov.in`
- **Protocol:** Socket.IO (websocket + polling transports), path `/socket.io`
- **Auth:** passed in the Socket.IO handshake payload, not an HTTP header:

```
io(socketUrl, { auth: { authorization: this.authToken } })
```

Outbound events

Event	Payload
`start`	`[{ taskType: "asr", config: { language: { sourceLanguage }, samplingRate: 16000, postProcessors: ["punctuation"] } }]` — international languages instead set `serviceId: "bhashini/intl/asr"`
(binary)	raw PCM16 audio frames, emitted continuously after `ready`

Inbound events

Event	Meaning
`connect`	socket connected
`ready`	server ready to receive audio
`response`	`{ pipelineResponse: [{ output: [{ source: <text> }] }], meta: { isIntermediateResult } }`
`disconnect` / `error` / `connect_error`	connection lifecycle

Call sites

- `src/index.html:10146-10211` — `createSocket` (socket factory)
- `src/index.html:10213-10237` — `initializeSocket`
- `src/index.html:10242-10284` — `scheduleReconnect`
- `src/index.html:10285-10305` — `handlePipelineConfig` (emits `start`)
- `src/index.html:12641-12680` — `initializeSocketForFileUpload` (second socket instance, same protocol, used for chunked file-upload streaming)

- `bhashini-streaming-app/lib/BhashiniStreamingASR.js:16,84-148` — standalone reusable client library, same protocol
- `bhashini-streaming-app/server.js:18-65` — local Express/`ws` relay server (browser mic → this Node server → Dhruva over Socket.IO → back to browser; keeps the Dhruva key server-side instead of in the browser)
- `asr_client.py`, `asr_recorder.py:12` — Python test clients (`socketio.Client`)
- `asr-stream-test.html:76` — manual browser test harness

---

2. BHASHINI / Dhruva — REST Pipeline API (ASR batch + Translation/NMT)

- **URL:** `https://dhruva-api.bhashini.gov.in/services/inference/pipeline`
- **Method:** `POST`
- **Headers:**

```
Accept: application/json, text/plain, */*
Content-Type: application/json
Authorization: <token>      // raw token, not "Bearer <token>"
```

- Translation calls use `this.translationAuthToken`
- ASR file-upload calls use `this.authToken`

Translation request

```
{
  "pipelineTasks": [{
    "taskType": "translation",
    "config": {
      "language": { "sourceLanguage": "hi", "targetLanguage": "en" },
      "serviceId": "ai4bharat/indictrans-v2-all-gpu--t4"
    }
  }],
  "inputData": { "input": [{ "source": "<text>" }] }
}
```

`serviceId` is `ai4bharat/indictrans-v2-all-gpu--t4` for Indic↔Indic/English, or `bhashini/intl/nmt` for international languages.

Response: `{ pipelineResponse: [{ output: [{ target: "<translated text>" }] }] }`

ASR (file upload) request

```
{
  "pipelineTasks": [{
    "taskType": "asr",
    "config": {
      "language": { "sourceLanguage": "hi" },
      "serviceId": "<getAsrServiceId(lang)>",
      "postProcessors": ["punctuation", "itn", "denoiser"],
      "samplingRate": 16000
    }
  }],
  "inputData": { "audio": [{ "audioContent": "<base64 PCM16>" }] }
}
```

International languages omit `postProcessors` and force `serviceId: "bhashini/intl/asr"`.

Response: `{ pipelineResponse: [{ output: [{ source: "<transcript>" }] }] }`

Call sites

- `src/index.html:5024-5046` — `_batchTranslateMoM` (batch-translates Minutes-of-Meeting text; batches of 25, 5 parallel groups)
- `src/index.html:8725-8764` — `requestTranslation` (live caption translation)
- `src/index.html:11830-11862` — `processSingleAudioCall` (whole-file ASR in one call)
- `src/index.html:~12181` — chunked ASR variant (same shape, for long files processed in pieces)
- `src/index.html:10737` — `getAsrServiceId()` helper
- `src/index.html:3607-3608` — `intlAsrServiceId` / `intlTranslationServiceId` config

---

3. BHASHINI / Dhruva — Audio Language Detection (ALD)

- **URL:** `https://dhruva-api.bhashini.gov.in/services/inference/audiolangdetection`
- **Method:** `POST`
- **Headers:** `Accept: /`, `Content-Type: application/json`, `Authorization: <ALD-specific key, hardcoded separately from authToken/translationAuthToken>`
- **Body:**

```
{
  "config": { "serviceId": "bhashini/ald" },
  "audio": [{ "audioContent": "<base64 PCM16>" }]
}
```

- **Response:** `{ output: [{ langPrediction: [{ langCode }] }] }`

Call sites

- `src/index.html:9412-9477` — `detectLanguageAndStartSocket` (initial detection before opening the streaming socket)
- `src/index.html:9522-9589` — `checkForLanguageSwitch` (periodic re-detection for "Dynamic Language Switch", driven by `feedRollingLanguageCheck`)

---

4. BHASHINI / Dhruva — LLM Inference (Gemma, for Minutes-of-Meeting)

- **URL:** `https://dhruva-api.bhashini.gov.in/services/inference/llm`
- **Method:** `POST`
- **Headers:** `Content-Type: application/json`, `Authorization: this.authToken`
- **Body:**

```
{
  "pipelineTasks": [{
    "taskType": "llm",
    "config": { "serviceId": "bhashini/google/gemma-4-26B-A4B-it", "temperature": 0 }
  }],
  "inputData": {
    "input": [[
      { "role": "system", "content": "<system prompt>" },
      { "role": "user", "content": "<text>" }
    ]]
  }
}
```

- **Response:** `result.pipelineResponse[0].output[0].target`
- **Call site:** `src/index.html:4213-4257` — `callGemmaAPI` (generates the MoM text)

---

5. BHASHINI / Dhruva — Connectivity Probe

- **URL:** `https://dhruva-api.bhashini.gov.in/`
- **Call:** `fetch(PROBE_URL, { mode: "no-cors", cache: "no-store" })`, every 15s
- Response is opaque (no-cors) — only round-trip success/latency is measured, feeds the network-status indicator in the UI.
- **Call site:** `src/index.html:6308-6345` — `setupConnectivityIndicator`

---

6. NVIDIA Cloud Functions (NVCF) — Alternative Translation Engine (Gemma 12B)

- **URL:** `https://68b3558f-f55a-41f8-b66b-c70e101fc770.invocation.api.nvcf.nvidia.com/v1/chat/completions` (`this.nvcfChatUrl`)
- **Method:** `POST` — OpenAI-compatible chat-completions schema
- **Auth:** `Authorization: Bearer <this.nvcfApiKey>`
- **Body:**

```
{
  "model": "gemma-4-12b",
  "messages": [ ... ],
  "temperature": 0.1,
  "top_p": 0.9,
  "max_tokens": 1024,
  "stream": false,
  "chat_template_kwargs": { "enable_thinking": false }
}
```

- **Response:** OpenAI-style `{ choices: [{ message: { content } }] }`

Routing note

Because `index.html` is loaded over `file://`, Chromium's CORS preflight to NVCF returns 401 from the renderer. So in Electron this call is relayed through the main process instead of being made directly:

- Renderer: `window.electronAPI.nvcfChat(...)` → `ipcRenderer.invoke('NVCF_CHAT', ...)` — `src/preload.js:31`
- Main process: `ipcMain.handle('NVCF_CHAT', ...)` via Node `https.request`, allow-listing only hosts matching `/(^|\.)(nvcf\.nvidia\.com|api\.nvidia\.com)$/i` — `src/index.js:528-587`
- Browser-direct fallback (non-Electron context): `src/index.html:8902-8957` — `callNvcfChat`

Feature flag: `this.useNvcfForTranslation` (off by default; Jenkins/Actions param `alt_translation_gemma`)

---

7. OpenAI — Alternative Translation Engine (Chat Completions)

- **URL:** `https://api.openai.com/v1/chat/completions` (`this.openaiTranslateUrl`)
- **Method:** `POST`
- **Headers:** `Content-Type: application/json`, `Authorization: Bearer <this.openaiApiKey>`
- **Model:** `gpt-4.1` (`this.openaiTranslateModel`)
- **Body:**

```
{
  "model": "gpt-4.1",
  "messages": [
    { "role": "system", "content": "<interpreterSystemPrompt>" },
    { "role": "user", "content": "<text>" }
  ],
  "temperature": 0.1,
  "top_p": 0.9,
  "max_tokens": 1024
}
```

- **Response:** `data.choices[0].message.content`
- Called directly from the renderer — OpenAI's CORS policy allows the `Authorization` header, so no IPC relay is needed (unlike NVCF).
- **Call site:** `src/index.html:8964-9009+` — `translateWithOpenAi`; config at `src/index.html:3689-3693`
- **Feature flag:** `this.useOpenAiForTranslation` (off by default; param `alt_translation_openai`)

---

8. OpenAI Realtime API — Alternative ASR Engine (WebSocket)

- **URL:** `wss://api.openai.com/v1/realtime?intent=transcription` (`this.altAsrUrl`)
- **Auth:** API key passed via WebSocket subprotocols, not a header:

```
["realtime", "openai-insecure-api-key.<key>"]
```

(A WS upgrade is exempt from CORS preflight, so — unlike NVCF — this runs directly from the renderer.)

- **Key:** `this.altAsrKey = this.openaiApiKey` (same key as #7)

- **Model:** `gpt-live-transcribe`, 24kHz PCM input (`audio/pcm`); captured audio is 16kHz and upsampled before sending.

Outbound messages

```
{ "type": "session.update", "session": { "type": "transcription", "audio": { "input": { "format": { . .. }, "transcription": { "model": "gpt-live-transcribe", "language": "...", "prompt": "..." } } } } }
```

then repeated:

```
{ "type": "input_audio_buffer.append", "audio": "<base64 PCM16>" }
```

Inbound messages

- `session.updated` (ready)
- transcript delta/completed events

Call site: `src/index.html:9643-9770+` ("ALT_ASR_ENGINE" block); config at `src/index.html:3610-3659`

Feature flag: `this.useAltAsr` (off by default; param `alt_asr`)

---

9. Web3Forms — Session Report Email Delivery

- **URL:** `https://api.web3forms.com/submit`
- **Method:** `POST`, `multipart/form-data`
- **Fields:** `access_key` (`this.web3formsKey` — a Web3Forms *public* access key, not treated as a secret by Web3Forms), `subject`, `from_name`, `message` (full HTML session report), `to` (hardcoded recipients: `shrutlekh.dibd@gmail.com,shailendra.dic@digitalindia.gov.in`)
- **Response:** `{ success: bool, message }`
- **Call site:** `src/index.html:8607-8635` — sends the HTML session/MoM report by email at end of session
- Effectively disabled if `web3formsKey` is blank (configurable via Jenkins param `web3forms_key`)

---

10. Google Fonts (static assets)

Not an API call — webfont stylesheet/preconnect links only.

- `https://fonts.googleapis.com`, `https://fonts.gstatic.com` — `src/index.html:8-16` and a duplicated `<head>` block at `src/index.html:7166-7168` (used by an exported/printable view)
- `https://cdn.socket.io/3.1.3/socket.io.min.js` — referenced as a string inside a generated HTML export at `src/index.html:2952`; the main app itself bundles Socket.IO locally (`src/lib/socket.io.min.js`)

---

11. GitHub Actions (build automation — no runtime API calls)

No code in this repo calls `api.github.com` or uses `octokit`. GitHub Actions is used declaratively only:

- `.github/workflows/build-for-user.yml` — `workflow_dispatch` trigger taking `user_name`, `api_key`, feature-flag inputs; resolves the Bhashini key (input or `secrets.BHASHINI_API_KEY`), masks it in logs, runs `node`

`build-for-user.js "<user_name>" "<key>"`, uploads the resulting exe as a build artifact.

- `.github/workflows/deploy-pages.yml` — deploys `gh-pages/` to GitHub Pages (`workflow_dispatch`/push triggered).

`build-server.js` / `build-ui.js` run a small **local** server (`http://localhost:3456` by default) with its own internal routes `/api/login`, `/api/history`, `/api/build`, `/api/builds`, used by the local build dashboard to shell out to `npm run make:win`. These are self-hosted endpoints, not external API calls.

---

12. Jenkins (build automation — no runtime API calls)

`Jenkinsfile` defines a declarative pipeline (agent `windows-builder`) mirroring the GitHub Actions workflow: same parameters (`user_name`, `api_key` as a masked Jenkins `password` param, feature-flag booleans, `web3forms_key`), falling back to Jenkins credential `BHASHINI_API_KEY` if no key is supplied, then running `build-for-user.js` and archiving the exe artifact.

No code calls the Jenkins REST/remote-trigger API — Jenkins is invoked externally via its own UI/webhook, not from within this codebase. `jenkins/` also contains `docker-compose.yml`, `nginx.conf`, `plugins.txt`, `setup-windows-agent.ps1` (infra/ops config only).

---

13. Key Management — `swap-key.js`

Not a network call. A local Node CLI that regex-replaces the hardcoded `authToken` / `translationAuthToken` string literals directly inside `src/index.html` before a build (`--show`, `--restore`, or `node swap-key.js <key>`). Default key values live in `ORIGINAL_ASR_KEY` / `ORIGINAL_TRANSLATION_KEY` constants at `swap-key.js:24-25`.

Credential Inventory

All of these are embedded as plaintext string literals in the shipped `src/index.html` rather than loaded from env/secret storage at runtime. The build pipelines (Jenkins/GitHub Actions) do support injecting a key at build time via `swap-key.js` / `build-for-user.js`, but the committed source always carries a live-looking key by default.

Key purpose	Variable	Location
Dhruva ASR streaming + file-upload + LLM token	`this.authToken`	`src/index.html:3773-3774`
Dhruva translation token	`this.translationAuthToken`	`src/index.html:3519-3520`
Dhruva ALD (language detection) token	inline string literal	`src/index.html:9429-9430`, `9540-9541`
NVIDIA NVCF API key	`this.nvcfApiKey`	`src/index.html:3674-3675`
OpenAI API key (shared by alt-ASR and alt-translation)	`this.openaiApiKey`	`src/index.html:3614-3615`
Web3Forms access key	`this.web3formsKey`	`src/index.html:3521`
Dhruva token for standalone streaming demo server	`AUTH_TOKEN`	`bhashini-streaming-app/server.js:21-22`